

检查 mcp 情况

检查当前mcp服务器状态

Thought for 1s (ctrl+o to expand)

● listMcpResources(List all MCP resources)

└ (No resources found)

● Bash(cat ~/.claude/settings.json 2>/dev/null | python3 -c "import sys,json; d=json.load(sys.stdin); print(json.dumps(d.get('mcpServers', d.get('mcp', {})), indent=2,...))"

└ {

"code-analyzer": {

"command": "/opt/anaconda3/bin/python3.12",

~+0 lines (ctrl+o to expand)

}

~+0 lines (ctrl+o to expand)

● Bash(cat /Users/a/.claude/settings.json 2>/dev/null | python3 -c "import sys,json; d=json.load(sys.stdin); print(json.dumps(d.get('mcpServers', d.get('mcp', {})), in...))"

└ {

"code-analyzer": {

"command": "/opt/anaconda3/bin/python3.12",

~+0 lines (ctrl+o to expand)

}

~+0 lines (ctrl+o to expand)

Thought for 4s (ctrl+o to expand)

● Bash(ps aux | grep -i code_analyzer | grep -v grep)

└ a

14565 0.0 0.3 419623856 73168 s000 S+ 10:24上午 0:00.30 /opt/anaconda3/bin/python3.12 /Users/a/Desktop/csp/week_12/code_analyzer_server.py

a 14158 0.0 0.3 419492784 74544 s000 S+ 10:19上午 0:00.30 /opt/anaconda3/bin/python3.12 /Users/a/Desktop/csp/week_12/code_analyzer_server.py

Thought for 1s, listed 1 directory (ctrl+o to expand)

当前 MCP 服务器状态

已配置的 MCP 服务器: code-analyzer

项目	详情
名称	code-analyzer
Python	/opt/anaconda3/bin/python3.12
服务器文件	/Users/a/Desktop/csp/week_12/code_analyzer_server.py
框架	FastMCP
json_response	true
进程状态	运行中 (2 个进程 PID: 14158, 14565)

提供的工具 (3 个) :

- analyze_file - 分析单个代码文件 (行数 / 函数结构 / 复杂度 / imports)

- scan_directory - 扫描目录内的代码文件概览

- compare_files - 对比两个文件的结构差异

资源: 无

该服务器未注册任何 MCP 资源 (仅提供工具) 。

~~~

总结: 当前只有 1 个本地 stdio 模式的 MCP 服务器 code-analyzer, 基于 FastMCP 构建, 用于代码质量分析。服务器正在正常运行。配置同时存在于全局和项目级 settings.json 中 (内容一致)。你想对这个服务器做什么操作吗?

● Sautéd for 17s

使用此代码分析工具 分析我该路径下所有的cpp代码

Thought for 6s (ctrl+o to expand)

你提到"该路径", 我猜你可能是指 /Users/a/Desktop/csp/week\_12/ (即 code-analyzer MCP 服务器所在的目录), 但也可能是当前工作目录 /Users/a。

能确认一下你要分析的是哪个路径吗? 可以直接告诉我具体路径, 例如:  
- /Users/a/Desktop/csp/week\_12/  
- 或者其他目录

● Baked for 8s

/Users/a/Desktop/csp/week\_10

Thought for 2s (ctrl+o to expand)

好的, 先用 scan\_directory 找出该目录下所有的 C++ 文件。

Thought for 1s, called code-analyzer (ctrl+o to expand)

扫描到 5 个 .cpp 文件。现在逐个深度分析:

Called code-analyzer 5 times (ctrl+o to expand)

all

Thought for 3s (ctrl+o to expand)

/Users/a/Desktop/csp/week\_10/ 全部 C++ 代码分析报告

~~~

📊 全局概览

#	文件	大小	总行	代码行	注释行	注释率	函数数	总复杂度	平均复杂度
1	zy_1.cpp	0.5 KB	26	21	3	11.54%	3	7	2.33
2	zy_2.cpp	1.0 KB	49	37	6	12.24%	3	10	3.33
3	zy_3.cpp	1.5 KB	65	50	9	13.85%	8	29	3.62
4	zy_4.cpp	1.5 KB	78	67	3	3.85%	7	23	3.29
5	zy_5.cpp	1.7 KB	90	78	4	4.44%	6	20	3.33
合计		6.3 KB	308	253	25	8.12%	27	89	3.30

~~~

1. `zy_1.cpp` - 疫情模拟（滑动窗口均值）

| 指标  | 值                                                                    |
|-----|----------------------------------------------------------------------|
| 复杂度 | 总计 7, 平均 2.33 (最低)                                                   |
| 函数  | <code>main</code> (C=2), 2 个 <code>for</code> 循环 (C=3, C=2)          |
| 依赖  | <code>iostream</code> , <code>algorithm</code> , <code>vector</code> |

| 代码最短，逻辑简单 - 对数组做邻近均值平滑处理。

2. `zy_2.cpp` - 时间段重叠计算（排序+双指针）

| 指标  | 值                                                                            |
|-----|------------------------------------------------------------------------------|
| 复杂度 | 总计 10, 平均 3.33                                                               |
| 函数  | <code>cmp</code> (C=3), <code>main</code> (C=3), <code>while</code> 循环 (C=4) |
| 依赖  | <code>iostream</code> , <code>algorithm</code> , <code>vector</code>         |

| 结构体 `Load`，区间排序后双指针扫描重叠时长。最内层 `while` 的 C=4 来自 `start < end` 条件分支。

3. `zy_3.cpp` - Wormholes 负环检测（Bellman-Ford）

| 指标  | 值                                                                                                                              |
|-----|--------------------------------------------------------------------------------------------------------------------------------|
| 复杂度 | 总计 29, 平均 3.62 (最高)                                                                                                            |
| 函数  | <code>hasNegativeCycle</code> (C=3), 嵌套 <code>for/if</code> (C=4,5,4), <code>main</code> (C=4), <code>while+for</code> (C=3×3) |
| 依赖  | <code>iostream</code> , <code>vector</code> , <code>climits</code>                                                             |

| 最复杂的文件。三层嵌套逻辑（`for` 松弛轮数 → `for` 遍历边 → `if` 松弛判断 + 负环检测）。

4. zy\_4.cpp - 双向最短路 (Dijkstra 正反图)

| 指标  | 值                                                          |
|-----|------------------------------------------------------------|
| 复杂度 | 总计 23, 平均 3.29                                             |
| 函数  | dijkstra (C=3), 3 个 for (C=3,4,2), main (C=4), while (C=4) |
| 依赖  | iostream, vector, queue                                    |

| 代码最长 (78 行有效代码), 但注释率最低 (3.85%)。正/反向图各跑一次 Dijkstra 求最短路径和。

5. zy\_5.cpp - 二分答案 + Dijkstra (最小容量约束)

| 指标  | 值                                                               |
|-----|-----------------------------------------------------------------|
| 复杂度 | 总计 20, 平均 3.33                                                  |
| 函数  | check (C=3), if (C=3), main (C=4), 2 个 while (C=4,3), for (C=3) |
| 依赖  | iostream, vector, queue, algorithm                              |

| 依赖最多 (5 个头文件)。二分搜索容量上限 + Dijkstra 可行性检查。两个 while 分别来自外层测试用例数和内层二分。

🔑 总结

| 维度   | 评价                                            |
|------|-----------------------------------------------|
| 总体质量 | 5 个文件均为算法竞赛风格代码, 结构清晰, 复杂度适中                  |
| 注释率  | 整体偏低 (平均 8.12%), zy_4 和 zy_5 尤为不足             |
| 最复杂  | zy_3.cpp - Bellman-Ford 负环检测, 含深层嵌套           |
| 最简洁  | zy_1.cpp - 仅 21 行有效代码, 复杂度最低                  |
| 共同依赖 | 全部用了 iostream + using namespace std, STL 使用广泛 |

调用正常

附上所有代码

.....

MCP 工具: 代码质量分析器 (

提供 3 个 tools:

- 1) analyze\_file      分析单文件 (行数 / 函数结构 / 复杂度 / imports)
- 2) scan\_directory    扫描目录内的代码文件概览
- 3) compare\_files     对比两个文件的结构差异

依赖:

```
pip install "mcp[cli]"
```

运行:

```
python code_analyzer_server.py
```

接入 (示例):

```
见同目录 mcp.json
```

.....

```
from __future__ import annotations
```

```

import ast
import json
import os
import re
from pathlib import Path
from typing import Any, Optional

from mcp.server.fastmcp import FastMCP

mcp = FastMCP(
    "code-analyzer",
    instructions=(
        "代码质量分析工具：用于分析单个代码文件的行数统计、函数/类结构、"
        "简化圈复杂度估计、以及导入依赖；也支持扫描目录与对比两个文件。"
    ),
    # 让 Host 直接拿到 JSON (dict/list) 而不是纯文本
    json_response=True,
)

# -----
# Tool 1: analyze_file
# -----
@mcp.tool(
    description=(
        "分析单个代码文件的质量指标。"
        "metrics 支持: lines, functions, complexity, imports; 传 all 表示全部。"
    )
)
def analyze_file(file_path: str, metrics: str = "all") -> dict[str, Any]:
    path = Path(file_path)
    if not path.exists():
        return {"error": f"文件不存在: {file_path}"}
    if not path.is_file():
        return {"error": f"路径不是文件: {file_path}"}

    content = _read_text_file(path)
    if content is None:
        return {"error": f"无法读取/解码文件: {file_path}"}

    ext = path.suffix.lower()
    lines = content.splitlines()

```

```

metric_list = (
    ["lines", "functions", "complexity", "imports"]
    if metrics.strip().lower() == "all"
    else [m.strip().lower() for m in metrics.split(",") if m.strip()]
)
valid = {"lines", "functions", "complexity", "imports"}
requested = [m for m in metric_list if m in valid]

result: dict[str, Any] = {
    "file": str(path.resolve()),
    "extension": ext,
    "size_bytes": path.stat().st_size,
    "metrics": requested,
}

if "lines" in requested:
    result["lines"] = _analyze_lines(lines)

functions: list[dict[str, Any]] = []
if "functions" in requested or "complexity" in requested:
    functions = _extract_functions(content, ext)

if "functions" in requested:
    result["functions"] = functions

if "complexity" in requested:
    func_only = [f for f in functions if f.get("kind") in ("function", "async_function")]
    for fn in func_only:
        fn["cyclomatic_complexity"] = _estimate_complexity(fn.get("body", ""))

    result["function_complexities"] = [
        {"name": f.get("name"), "line": f.get("line"), "complexity":
f.get("cyclomatic_complexity")}
        for f in func_only
    ]
    total = sum(int(f.get("cyclomatic_complexity") or 0) for f in func_only)
    result["total_complexity"] = total
    result["avg_complexity"] = round(total / len(func_only), 2) if func_only else 0

if "imports" in requested:
    result["imports"] = _extract_imports(content, ext)

return result

```

```

# -----
# Tool 2: scan_directory
# -----
@mcp.tool(
    description=(
        "扫描指定目录，统计常见代码文件的基本信息（路径、大小、行数概览）。"
        "可通过 extensions 指定扩展名（逗号分隔），并用 max_files 限制扫描数量。"
    )
)
def scan_directory(
    directory_path: str,
    extensions: str = ".py,.js,.ts,.jsx,.tsx,.java,.cpp,.h,.hpp,.cs,.go,.rs,.rb,.php,.swift,.kt,.scala",
    max_files: int = 2000,
) -> dict[str, Any]:
    root = Path(directory_path)
    if not root.exists():
        return {"error": f"目录不存在：{directory_path}"}
    if not root.is_dir():
        return {"error": f"路径不是目录：{directory_path}"}

    ext_list = [
        e.strip().lower() if e.strip().startswith(".") else f".{e.strip().lower()}"
        for e in extensions.split(",")
        if e.strip()
    ]

    skip_dirs = {
        "node_modules",
        ".git",
        "__pycache__",
        ".venv",
        "venv",
        ".idea",
        ".vscode",
        "build",
        "dist",
        ".next",
        ".nuxt",
        "target",
        "bin",
        "obj",
        ".workbuddy",
    }

```

```

skip_dirs_lower = {d.lower() for d in skip_dirs}

files: list[dict[str, Any]] = []
total_bytes = 0
scanned = 0

try:
    for current_root, dirs, filenames in os.walk(root):
        dirs[:] = [d for d in dirs if d.lower() not in skip_dirs_lower]

        for filename in filenames:
            if scanned >= max_files:
                break
            ext = os.path.splitext(filename)[1].lower()
            if ext not in ext_list:
                continue
            full_path = Path(current_root) / filename
            scanned += 1

            try:
                size = full_path.stat().st_size
                total_bytes += size

                content = _read_text_file(full_path)
                line_info = _analyze_lines(content.splitlines()) if content is not None
            except:
                pass

            files.append(
                {
                    "relative_path": str(full_path.relative_to(root)),
                    "extension": ext,
                    "size_bytes": size,
                    "size_kb": round(size / 1024, 1),
                    "lines": line_info,
                }
            )
        except (OSError, PermissionError):
            continue
        if scanned >= max_files:
            break
except PermissionError:
    return {"error": f"无权限访问目录: {directory_path}"}

by_ext: dict[str, int] = {}

```

```

for f in files:
    by_ext[f["extension"]] = by_ext.get(f["extension"], 0) + 1

return {
    "directory": str(root.resolve()),
    "max_files": max_files,
    "scanned_files": scanned,
    "matched_files": len(files),
    "total_size_kb": round(total_bytes / 1024, 1),
    "by_extension": [
        {"extension": ext, "count": count}
        for ext, count in sorted(by_ext.items(), key=lambda x: -x[1])
    ],
    "files": sorted(files, key=lambda x: -x["size_bytes"]),
}

```

```

# -----
# Tool 3: compare_files
# -----
@mcp.tool(
    description="对比两个代码文件的结构差异（行数、函数/类数量、imports 数量、体积等）。"
)
def compare_files(file_a: str, file_b: str) -> dict[str, Any]:
    info_a = _quick_file_info(file_a)
    info_b = _quick_file_info(file_b)
    if not info_a or not info_b:
        missing = []
        if not info_a:
            missing.append(file_a)
        if not info_b:
            missing.append(file_b)
        return {"error": f"以下文件无法读取：{'、'.join(missing)}"}

    diff = {
        "size_bytes": info_b["size_bytes"] - info_a["size_bytes"],
        "total_lines": info_b["total_lines"] - info_a["total_lines"],
        "code_lines": info_b["code_lines"] - info_a["code_lines"],
        "comment_lines": info_b["comment_lines"] - info_a["comment_lines"],
        "blank_lines": info_b["blank_lines"] - info_a["blank_lines"],
        "func_count": info_b["func_count"] - info_a["func_count"],
        "class_count": info_b["class_count"] - info_a["class_count"],
        "import_count": info_b["import_count"] - info_a["import_count"],
    }

```

```

    }

    return {
        "file_a": info_a,
        "file_b": info_b,
        "difference": diff,
        "summary": (
            f"文件 A [{Path(file_a).name}] vs 文件 B [{Path(file_b).name}]: "
            f"B 比 A {'大' if diff['size_bytes'] >= 0 else '小'} {abs(diff['size_bytes'])} 字节; "
            f"代码行 {'多' if diff['code_lines'] >= 0 else '少'} {abs(diff['code_lines'])}; "
            f"函数 {'多' if diff['func_count'] >= 0 else '少'} {abs(diff['func_count'])}; "
            f"类 {'多' if diff['class_count'] >= 0 else '少'} {abs(diff['class_count'])}"
        ),
    }
}

# -----
# Helpers
# -----

def _read_text_file(path: Path) -> Optional[str]:
    try:
        return path.read_text(encoding="utf-8")
    except UnicodeDecodeError:
        try:
            return path.read_text(encoding="gbk")
        except Exception:
            return None
    except Exception:
        return None

def _analyze_lines(lines: list[str]) -> dict[str, Any]:
    total = len(lines)
    code = 0
    comments = 0
    blank = 0

    for line in lines:
        s = line.strip()
        if not s:
            blank += 1
            continue

    # 仅做“行级”粗略判断（多行注释块不做完整语法分析）

```

```

        if s.startswith(("#", "//", "/*", "*", "--", "%")):
            comments += 1
        else:
            code += 1

    return {
        "total_lines": total,
        "code_lines": code,
        "comment_lines": comments,
        "blank_lines": blank,
        "comment_ratio": f'{round(comments / max(total, 1) * 100, 2)}%',
    }

def _extract_functions(content: str, ext: str) -> list[dict[str, Any]]:
    if ext == ".py":
        return _extract_python_functions(content)

    functions: list[dict[str, Any]] = []
    patterns = [
        (r"(?:function\s+)?(\w+)\s*\([^\)]*\)\s*\{", "function"), # JS
        (r"(?:def|fun|func|fn)\s+(\w+)\s*\(", "function"), # 多语言
        (
            r"(?:public|private|protected)?\s*(?:static\s+)?(\w+)\s*\([^\)]*\)\s*(?:throws\s+\w+)?\s*\{",
            "method",
        ), # Java/C#
        (r"class\s+(\w+)", "class"),
    ]
    for pattern, kind in patterns:
        for match in re.finditer(pattern, content):
            line_no = content[: match.start()].count("\n") + 1
            start = max(0, match.start() - 80)
            snippet = content[start : match.end() + 200]
            functions.append(
                {
                    "name": match.group(1),
                    "line": line_no,
                    "kind": kind,
                    "body": snippet,
                }
            )
    return functions

```

```

def _extract_python_functions(content: str) -> list[dict[str, Any]]:
    functions: list[dict[str, Any]] = []
    try:
        tree = ast.parse(content)
        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
                src = _get_node_source(content, node)
                functions.append(
                    {
                        "name": node.name,
                        "line": node.lineno,
                        "kind": "function",
                        "args_count": len(node.args.args),
                        "has_docstring": ast.get_docstring(node) is not None,
                        "body": src,
                    }
                )
            elif isinstance(node, ast.AsyncFunctionDef):
                src = _get_node_source(content, node)
                functions.append(
                    {
                        "name": node.name,
                        "line": node.lineno,
                        "kind": "async_function",
                        "args_count": len(node.args.args),
                        "has_docstring": ast.get_docstring(node) is not None,
                        "body": src,
                    }
                )
            elif isinstance(node, ast.ClassDef):
                functions.append(
                    {
                        "name": node.name,
                        "line": node.lineno,
                        "kind": "class",
                        "methods": sum(
                            1
                            for n in node.body
                            if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef))
                        ),
                        "has_docstring": ast.get_docstring(node) is not None,
                    }
                )
    
```

```

except SyntaxError:
    # 语法错误时回退到通用正则
    return _extract_functions(content, ext=".txt")

return functions

```

```

def _get_node_source(content: str, node: Any) -> str:
    lines = content.splitlines()
    start = max(0, int(getattr(node, "lineno", 1)) - 1)
    end = int(getattr(node, "end_lineno", start + 1))
    end = min(len(lines), max(end, start + 1))
    return "\n".join(lines[start:end])

```

```

def _estimate_complexity(body: str) -> int:
    """
    估算圈复杂度（简化 McCabe）：
    基线 1 + 控制流/布尔分支关键词计数。
    """
    if not body:
        return 1
    complexity = 1
    keywords = [
        r"\bif\b",
        r"\belif\b",
        r"\bfor\b",
        r"\bwhile\b",
        r"\bcase\b",
        r"\bcatch\b",
        r"\bexcept\b",
        r"\bwhen\b",
        r"\band\b",
        r"\bor\b",
        r"&&",
        r"\|",
        r"\?",
    ]
    for kw in keywords:
        complexity += len(re.findall(kw, body))
    return complexity

```

```

def _extract_imports(content: str, ext: str) -> list[dict[str, Any]]:

```

```
imports: list[dict[str, Any]] = []
```

```
if ext == ".py":
    try:
        tree = ast.parse(content)
        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    imports.append(
                        {"type": "import", "source": alias.name, "alias": alias.asname}
                    )
            elif isinstance(node, ast.ImportFrom):
                module = node.module or ""
                for alias in node.names:
                    imports.append(
                        {
                            "type": "from_import",
                            "source": module,
                            "name": alias.name,
                            "alias": alias.asname,
                        }
                    )
    except SyntaxError:
        for match in re.finditer(r"^(?:import|from)\s+(.+)$", content, re.MULTILINE):
            imports.append({"type": "statement", "raw": match.group(0).strip()})
    return imports
```

```
patterns = [
    (r"(?:import|require)\s+[\"](?:[^\"]+)[\"]", "import"), # JS/TS
    (r"(?:from|using|include|import)\s+(\w+(?:\.\w+)*)", "import_statement"),
    (r"#include\s*<[\"](?:[^\"]+)[\"]>", "include"), # C/C++
]
for pattern, kind in patterns:
    for match in re.finditer(pattern, content):
        imports.append({"type": kind, "value": match.group(1)})
return imports
```

```
def _quick_file_info(file_path: str) -> Optional[dict[str, Any]]:
    path = Path(file_path)
    if not path.exists() or not path.is_file():
        return None

    content = _read_text_file(path)
```

```
if content is None:
    return None
```

```
ext = path.suffix.lower()
lines = content.splitlines()
line_info = _analyze_lines(lines)
functions = _extract_functions(content, ext)
imports = _extract_imports(content, ext)

return {
    "file": str(path.resolve()),
    "extension": ext,
    "size_bytes": path.stat().st_size,
    "total_lines": line_info["total_lines"],
    "code_lines": line_info["code_lines"],
    "comment_lines": line_info["comment_lines"],
    "blank_lines": line_info["blank_lines"],
    "func_count": len([f for f in functions if f.get("kind") in ("function", "async_function")]),
    "class_count": len([f for f in functions if f.get("kind") == "class"]),
    "import_count": len(imports),
}
```

```
if __name__ == "__main__":
    # Stdio transport by default (适配 Claude Code / WorkBuddy / Cursor 等 Host)
    mcp.run()
```

mcp.json

```
{
  "mcpServers": {
    "code-analyzer": {
      "command": "/opt/anaconda3/bin/python3.12",
      "args": ["/Users/a/Desktop/csp/week_12/code_analyzer_server.py"],
      "env": {}
    }
  }
}
```